
MODULE *protocol*

Version 2 *Zcash P2P* protocol specification, following the draft *ZIP* “Version 2 *Zcash P2P* Network Protocol” (*zcash/zips* $\neq$ 1344).

Phase 1 models connection setup over the stream layer of *streams.tla*: the *init* handshake on the dedicated handshake stream, protocol version negotiation, and the long-lived block announcement streams, including the rule that at most one announcement stream of a type may be open per direction and the sender’s option to replace a finished or reset stream.

Communication keeps the legacy model’s message consumption discipline: a peer decides only from its own records; the only place both peers’ records are written together is the transport itself (connection open/close, stream open, and the freeing of a stream slot once both sides have closed it).

Two boolean constants select how a receiver interprets the draft:

StrictSingleton TRUE a second open announcement stream of the same type from a peer is a *PROTOCOL_ERROR* (literal reading of “Announcement Streams”)
 FALSE the receiver accepts it and drains both
RefusePreHandshake TRUE announcement streams that arrive before the local handshake completes are refused with *REFUSED* (“Connection Handshake”: MAY refuse)
 FALSE they are buffered until it completes

Under *StrictSingleton* the invariant *NoHonestProtocolError* is violated: two conformant peers disconnect because stream independence lets a replacement stream’s type byte arrive before the old stream’s *FIN* or reset.

See documents/*v2-modeling.md* for the full write-up.

EXTENDS *TLC*, *Naturals*, *Sequences*, *FiniteSets*, *streams*, *records*

CONSTANT <i>InitialPeers</i>	set of peers
CONSTANT <i>MaxBlock</i>	maximum block height (initial and mined)
CONSTANT <i>MaxRestarts</i>	finish/reset of own announcement streams, per peer pair
CONSTANT <i>MinVersion</i>	minimum protocol version of the draft (not yet assigned)
CONSTANT <i>Versions</i>	protocol versions a peer may advertise
CONSTANT <i>StrictSingleton</i>	see module comment
CONSTANT <i>RefusePreHandshake</i>	see module comment

VARIABLE *nodes*

vars $\triangleq$ $\langle nodes \rangle$

See *README* for an explanation of symmetry reduction.

Symmetry $\triangleq$ *Permutations*(*InitialPeers*)

For each initial peer construct a set of all other peers.

OtherPeers $\triangleq$ $[n \in \text{InitialPeers} \mapsto \text{InitialPeers} \setminus \{n\}]$

Min(*a*, *b*) $\triangleq$ IF *a* < *b* THEN *a* ELSE *b*

Max(*a*, *b*) $\triangleq$ IF *a* > *b* THEN *a* ELSE *b*

$$\begin{aligned}
Init &\triangleq \\
&\exists \textit{blockset} \in [\textit{InitialPeers} \rightarrow (1 \dots \textit{MaxBlock})] : \\
&\exists \textit{verset} \in [\textit{InitialPeers} \rightarrow \textit{Versions}] : \\
&\quad \textit{nodes} = [i \in \textit{InitialPeers} \mapsto [\\
&\quad \quad \textit{conn} \mapsto [j \in \textit{OtherPeers}[i] \mapsto \text{"none"}], \\
&\quad \quad \textit{close} \mapsto [j \in \textit{OtherPeers}[i] \mapsto \textit{NoCode}], \\
&\quad \quad \textit{streams} \mapsto [j \in \textit{OtherPeers}[i] \mapsto [sid \in \textit{StreamIds}(i, j) \mapsto \textit{NullStream}]], \\
&\quad \quad \textit{init_sent} \mapsto [j \in \textit{OtherPeers}[i] \mapsto \text{FALSE}], \\
&\quad \quad \textit{init_recvd} \mapsto [j \in \textit{OtherPeers}[i] \mapsto \text{FALSE}], \\
&\quad \quad \textit{version} \mapsto [j \in \textit{OtherPeers}[i] \mapsto 0], \\
&\quad \quad \textit{peer_tip} \mapsto [j \in \textit{OtherPeers}[i] \mapsto 0], \\
&\quad \quad \textit{announced} \mapsto [j \in \textit{OtherPeers}[i] \mapsto 0], \\
&\quad \quad \textit{restarts} \mapsto [j \in \textit{OtherPeers}[i] \mapsto 0], \\
&\quad \quad \textit{blocks} \mapsto 1 \dots \textit{blockset}[i], \\
&\quad \quad \textit{my_version} \mapsto \textit{verset}[i] \\
&\quad] \\
&]
\end{aligned}$$

— Helpers —

$\textit{Height}(n) \triangleq \textit{Cardinality}(\textit{nodes}[n].\textit{blocks})$

$\textit{Connected}(n, m) \triangleq \textit{nodes}[n].\textit{conn}[m] \in \{\text{"initiator"}, \text{"responder"}\}$

The handshake is complete for n once it has both sent and received an *init* record (draft: “Handshake Sequence”, step 3).

$\textit{HandshakeComplete}(n, m) \triangleq \textit{nodes}[n].\textit{init_sent}[m] \wedge \textit{nodes}[n].\textit{init_recvd}[m]$

$S(n, m, \textit{sid}) \triangleq \textit{nodes}[n].\textit{streams}[m][\textit{sid}]$

Slots n may use to open a new stream toward m (lowest free slot is taken).

$\textit{FreeSlots}(n, m) \triangleq \{k \in 1 \dots \textit{MaxStreams} : S(n, m, \langle n, k \rangle).status = \text{"none"}\}$

$\textit{FreeSlot}(n, m) \triangleq \text{CHOOSE } k \in \textit{FreeSlots}(n, m) : \forall j \in \textit{FreeSlots}(n, m) : k \leq j$

Announcement streams of type t that n currently sends to m on.

$\textit{LiveAnn}(n, m, t) \triangleq \{sid \in \textit{StreamIds}(n, m) : \\
\begin{aligned}
&\wedge sid[1] = n \\
&\wedge S(n, m, sid).status = \text{"open"} \\
&\wedge S(n, m, sid).rtype = t \\
&\wedge S(n, m, sid).out = \text{"open"}\}
\end{aligned}$

Handshake streams n knows about on its connection with m .

$\textit{HandshakeStreams}(n, m) \triangleq \{sid \in \textit{StreamIds}(n, m) : \\
\begin{aligned}
&\wedge S(n, m, sid).status = \text{"open"} \\
&\wedge S(n, m, sid).rtype = \textit{HandshakeType}\}
\end{aligned}$

Transport: once both peers have closed a stream its slot is freed.

$Settle(ns, n, m, sid) \triangleq$
 IF $\wedge ns[n].streams[m][sid].status = \text{"closed"}$
 $\wedge ns[m].streams[n][sid].status = \text{"closed"}$
 THEN $[ns \text{ EXCEPT } ![n].streams[m][sid] = \text{NullStream},$
 $![m].streams[n][sid] = \text{NullStream}]$
 ELSE ns

Transport: n closes the connection to m with an application error code.
 Both peers observe the close; data in flight is lost.

$Close(ns, n, m, code) \triangleq$
 $[ns \text{ EXCEPT } ![n].conn[m] = \text{"closed"},$
 $![n].close[m] = code,$
 $![n].streams[m] = [sid \in StreamIds(n, m) \mapsto \text{NullStream}],$
 $![m].conn[n] = \text{"closed"},$
 $![m].close[n] = code,$
 $![m].streams[n] = [sid \in StreamIds(n, m) \mapsto \text{NullStream}]]$

— Connection —

n dials m . The *QUIC* handshake is out of scope and establishment is atomic:
 n becomes the initiator and m the responder of the application handshake.

$Connect \triangleq$
 $\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\wedge nodes[n].conn[m] = \text{"none"}$
 $\wedge nodes[m].conn[n] = \text{"none"}$
 $\wedge nodes' = [nodes \text{ EXCEPT } ![n].conn[m] = \text{"initiator"},$
 $![m].conn[n] = \text{"responder"}]$

— Handshake —

The initiator opens the handshake stream and sends its *init* record on it
 (draft: “Handshake Sequence”, step 1). The type byte and the record are
 queued together: they travel in order on the same stream.

$OpenHandshakeStream \triangleq$
 $\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\wedge nodes[n].conn[m] = \text{"initiator"}$
 $\wedge \neg nodes[n].init_sent[m]$
 $\wedge FreeSlots(n, m) \neq \{\}$
 $\wedge \text{LET } sid \triangleq \langle n, FreeSlot(n, m) \rangle$
 IN $nodes' = [nodes \text{ EXCEPT }$
 $![n].streams[m][sid] = \text{OpenerStream}(\text{HandshakeType}),$

$$\begin{aligned}
! [m].streams[n][sid] &= PeerStream(HandshakeType, \\
&\quad \langle MakeInit(nodes[n].my_version, Height(n)) \rangle), \\
! [n].init_sent[m] &= TRUE]
\end{aligned}$$

The responder sends its own *init* record on the handshake stream. It may do so as soon as it has seen the stream's type byte, without waiting for the initiator's *init* record (draft: "Handshake Sequence").

SendInitResponder $\triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\exists sid \in HandshakeStreams(n, m) :$

$\wedge nodes[n].conn[m] = \text{"responder"}$

$\wedge \neg nodes[n].init_sent[m]$

$\wedge nodes' = [nodes \text{ EXCEPT}$

$! [m].streams[n][sid] = PutData(@, MakeInit(nodes[n].my_version, Height(n))),$

$! [n].init_sent[m] = TRUE]$

n consumes the type byte of a stream *m* opened. What happens depends on the type and on *n*'s own state:

0x00 from the responder, or a second 0x00 $\rightarrow PROTOCOL_ERROR$ ("Connection Handshake")
announcement before *n*'s handshake completed $\rightarrow$ refuse with *REFUSED*, or wait (*RefusePreHandshake*)
announcement duplicating an open one of type *t* $\rightarrow PROTOCOL_ERROR$, or accept (*StrictSingleton*)
otherwise $\rightarrow$ record the type

RecvTypeByte $\triangleq$

$\exists n \in InitialPeers :$

$\exists m \in OtherPeers[n] :$

$\exists sid \in StreamIds(n, m) :$

LET *s* $\triangleq S(n, m, sid)$

IN

$\wedge Connected(n, m)$

$\wedge s.status = \text{"open"}$

$\wedge s.rtype = \text{"unknown"}$

$\wedge Len(s.inq) > 0$

$\wedge IsTypeByte(Head(s.inq))$

$\wedge \text{LET } t \triangleq Head(s.inq).t$

$accept \triangleq [nodes \text{ EXCEPT } ! [n].streams[m][sid].rtype = t,$

$! [n].streams[m][sid].inq = Tail(@)]$

$legalHs \triangleq nodes[n].conn[m] = \text{"responder"} \wedge HandshakeStreams(n, m) = \{\}$

$dup \triangleq \exists o \in StreamIds(n, m) :$

$\wedge o \neq sid$

$\wedge o[1] = m$

$\wedge S(n, m, o).status = \text{"open"}$

$\wedge S(n, m, o).rtype = t$

IN

$\wedge t \in StreamTypes$

$$\begin{aligned}
& \wedge \vee \wedge t = \text{HandshakeType} \\
& \quad \wedge \vee \wedge \text{legalHs} \\
& \quad \quad \wedge \text{nodes}' = \text{accept} \\
& \quad \vee \wedge \neg \text{legalHs} \\
& \quad \quad \wedge \text{nodes}' = \text{Close}(\text{nodes}, n, m, \text{"PROTOCOL_ERROR"}) \\
& \vee \wedge t \in \text{AnnTypes} \\
& \quad \wedge \vee \wedge \neg \text{HandshakeComplete}(n, m) \\
& \quad \quad \wedge \text{RefusePreHandshake} \\
& \quad \quad \wedge \text{nodes}' = \text{Settle}([\text{nodes} \text{ EXCEPT} \\
& \quad \quad \quad ! [n].\text{streams}[m][\text{sid}] = \text{ClosedStream}, \\
& \quad \quad \quad ! [m].\text{streams}[n][\text{sid}] = \text{PutStop}(@, \text{"REFUSED"})], \\
& \quad \quad \quad n, m, \text{sid}) \\
& \quad \vee \wedge \text{HandshakeComplete}(n, m) \\
& \quad \quad \wedge \vee \wedge \text{StrictSingleton} \wedge \text{dup} \\
& \quad \quad \quad \wedge \text{nodes}' = \text{Close}(\text{nodes}, n, m, \text{"PROTOCOL_ERROR"}) \\
& \quad \quad \vee \wedge \neg (\text{StrictSingleton} \wedge \text{dup}) \\
& \quad \quad \quad \wedge \text{nodes}' = \text{accept}
\end{aligned}$$

n consumes m 's *init* record from the handshake stream (draft: "Handshake Validation"):

second *init* record $\rightarrow \text{PROTOCOL_ERROR}$

version below *MinVersion* $\rightarrow \text{OBSOLETE}$

otherwise $\rightarrow$ handshake received; negotiated *version* = *min*

$\text{RecvInit} \triangleq$

$\exists n \in \text{InitialPeers} :$

$\exists m \in \text{OtherPeers}[n] :$

$\exists \text{sid} \in \text{HandshakeStreams}(n, m) :$

LET $s \triangleq S(n, m, \text{sid})$

IN

$\wedge \text{Connected}(n, m)$

$\wedge \text{Len}(s.\text{inq}) > 0$

$\wedge \text{Head}(s.\text{inq}).\text{kind} = \text{"init"}$

$\wedge \text{LET } \text{msg} \triangleq \text{Head}(s.\text{inq})$

IN

$\vee \wedge \text{nodes}[n].\text{init_rcvd}[m]$

$\wedge \text{nodes}' = \text{Close}(\text{nodes}, n, m, \text{"PROTOCOL_ERROR"})$

$\vee \wedge \neg \text{nodes}[n].\text{init_rcvd}[m]$

$\wedge \text{msg.version} < \text{MinVersion}$

$\wedge \text{nodes}' = \text{Close}(\text{nodes}, n, m, \text{"OBSOLETE"})$

$\vee \wedge \neg \text{nodes}[n].\text{init_rcvd}[m]$

$\wedge \text{msg.version} \geq \text{MinVersion}$

$\wedge \text{nodes}' = [\text{nodes} \text{ EXCEPT}$

$! [n].\text{streams}[m][\text{sid}].\text{inq} = \text{Tail}(@),$

$! [n].\text{init_rcvd}[m] = \text{TRUE},$

$! [n].\text{version}[m] = \text{Min}(\text{nodes}[n].\text{my_version}, \text{msg.version}),$

$$![n].peer_tip[m] = msg.start_height]$$

— Announcement streams —

After its handshake completes, n opens one announcement stream of each type it has none open for (draft: “Announcement Streams”). This is also how a replacement is opened after a finish, reset, or refusal. A reset can overtake announcements still in flight, so the sender forgets what it announced and re-announces its tip on the new stream.

$OpenAnnouncementStream \triangleq$

$\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\exists t \in AnnTypes :$
 $\wedge Connected(n, m)$
 $\wedge HandshakeComplete(n, m)$
 $\wedge LiveAnn(n, m, t) = \{\}$
 $\wedge FreeSlots(n, m) \neq \{\}$
 $\wedge LET\ sid \triangleq \langle n, FreeSlot(n, m) \rangle$
 $IN\ nodes' = [nodes\ EXCEPT$
 $\quad ![n].streams[m][sid] = OpenerStream(t),$
 $\quad ![m].streams[n][sid] = PeerStream(t, \langle \rangle),$
 $\quad ![n].announced[m] = 0]$

n announces its current tip to m when it has grown since the last announcement (draft: “Block Announcements”, header announcement).

$SendAnnouncement \triangleq$

$\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\exists sid \in LiveAnn(n, m, BlockAnnType) :$
 $\wedge nodes[n].announced[m] < Height(n)$
 $\wedge nodes' = [nodes\ EXCEPT$
 $\quad ![m].streams[n][sid] = PutData(@, MakeHeaderAnnouncement(Height(n))),$
 $\quad ![n].announced[m] = Height(n)]$

n consumes a header announcement from m . Announcements are only processed once n 's handshake is complete (draft: “Connection Handshake”).

$RecvAnnouncement \triangleq$

$\exists n \in InitialPeers :$
 $\exists m \in OtherPeers[n] :$
 $\exists sid \in StreamIds(n, m) :$
 $LET\ s \triangleq S(n, m, sid)$
 IN
 $\wedge Connected(n, m)$
 $\wedge HandshakeComplete(n, m)$

$$\begin{aligned}
& \wedge s.status = \text{"open"} \\
& \wedge s.rtype \in \text{AnnTypes} \\
& \wedge \text{Len}(s.inq) > 0 \\
& \wedge \text{Head}(s.inq).kind = \text{"header"} \\
& \wedge nodes' = [nodes \text{ EXCEPT} \\
& \quad ! [n].streams[m][sid].inq = \text{Tail}(@), \\
& \quad ! [n].peer_tip[m] = \text{Max}(@, \text{Head}(s.inq).height)]
\end{aligned}$$

n finishes one of its announcement streams. The draft allows a sender to open a replacement afterwards; why it finished (backpressure, internal restart) is not modeled. Bounded by *MaxRestarts* to keep liveness meaningful.

FinishAnnouncementStream $\triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \exists sid \in \text{LiveAnn}(n, m, \text{BlockAnnType}) : \\
& \quad \quad \quad \wedge nodes[n].restarts[m] < \text{MaxRestarts} \\
& \quad \quad \quad \wedge nodes' = \text{Settle}([nodes \text{ EXCEPT} \\
& \quad \quad \quad \quad ! [n].streams[m][sid] = \text{ClosedStream}, \\
& \quad \quad \quad \quad ! [m].streams[n][sid] = \text{PutData}(@, \text{FIN}), \\
& \quad \quad \quad \quad ! [n].restarts[m] = @ + 1], \\
& \quad \quad \quad n, m, sid)
\end{aligned}$$

Same as *FinishAnnouncementStream* but via *RESET_STREAM*, which can overtake records still in flight on that stream.

ResetAnnouncementStream $\triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \exists sid \in \text{LiveAnn}(n, m, \text{BlockAnnType}) : \\
& \quad \quad \quad \wedge nodes[n].restarts[m] < \text{MaxRestarts} \\
& \quad \quad \quad \wedge nodes' = \text{Settle}([nodes \text{ EXCEPT} \\
& \quad \quad \quad \quad ! [n].streams[m][sid] = \text{ClosedStream}, \\
& \quad \quad \quad \quad ! [m].streams[n][sid] = \text{PutReset}(@, \text{"INTERNAL_ERROR"}), \\
& \quad \quad \quad \quad ! [n].restarts[m] = @ + 1], \\
& \quad \quad \quad n, m, sid)
\end{aligned}$$

— Stream teardown —

n consumes a *FIN*. A *FIN* before the type byte, or on the handshake stream, is handled per the draft (*PROTOCOL_ERROR*; graceful close) although honest peers in this phase never produce either.

RecvFin $\triangleq$

$$\begin{aligned}
& \exists n \in \text{InitialPeers} : \\
& \quad \exists m \in \text{OtherPeers}[n] : \\
& \quad \quad \exists sid \in \text{StreamIds}(n, m) :
\end{aligned}$$

$\text{LET } s \triangleq S(n, m, sid)$
 IN
 $\wedge \text{Connected}(n, m)$
 $\wedge s.\text{status} = \text{"open"}$
 $\wedge \text{Len}(s.\text{inq}) > 0$
 $\wedge \text{IsFin}(\text{Head}(s.\text{inq}))$
 $\wedge \vee \wedge s.\text{rtype} = \text{"unknown"}$
 $\quad \wedge \text{nodes}' = \text{Close}(\text{nodes}, n, m, \text{"PROTOCOL_ERROR"})$
 $\vee \wedge s.\text{rtype} = \text{HandshakeType}$
 $\quad \wedge \text{nodes}' = \text{Close}(\text{nodes}, n, m, \text{"NO_ERROR"})$
 $\vee \wedge s.\text{rtype} \in \text{AnnTypes}$
 $\quad \wedge \text{nodes}' = \text{Settle}([\text{nodes} \text{ EXCEPT } ![n].\text{streams}[m][sid] = \text{ClosedStream}],$
 $\quad \quad n, m, sid)$

n observes a *RESET_STREAM* from m . Queued data on that stream is discarded.

$\text{RecvReset} \triangleq$
 $\exists n \in \text{InitialPeers} :$
 $\exists m \in \text{OtherPeers}[n] :$
 $\exists sid \in \text{StreamIds}(n, m) :$
 $\text{LET } s \triangleq S(n, m, sid)$
 IN
 $\wedge \text{Connected}(n, m)$
 $\wedge s.\text{status} = \text{"open"}$
 $\wedge s.\text{in_reset} \neq \text{NoCode}$
 $\wedge \vee \wedge s.\text{rtype} = \text{HandshakeType}$
 $\quad \wedge \text{nodes}' = \text{Close}(\text{nodes}, n, m, \text{"NO_ERROR"})$
 $\vee \wedge s.\text{rtype} \neq \text{HandshakeType}$
 $\quad \wedge \text{nodes}' = \text{Settle}([\text{nodes} \text{ EXCEPT } ![n].\text{streams}[m][sid] = \text{ClosedStream}],$
 $\quad \quad n, m, sid)$

n observes a *STOP_SENDING* from m on a stream n is sending on, and resets its sending direction as the draft recommends ("Request Streams").

$\text{RecvStop} \triangleq$
 $\exists n \in \text{InitialPeers} :$
 $\exists m \in \text{OtherPeers}[n] :$
 $\exists sid \in \text{StreamIds}(n, m) :$
 $\text{LET } s \triangleq S(n, m, sid)$
 IN
 $\wedge \text{Connected}(n, m)$
 $\wedge sid[1] = n$
 $\wedge s.\text{status} = \text{"open"}$
 $\wedge s.\text{out} = \text{"open"}$
 $\wedge s.\text{stop} \neq \text{NoCode}$
 $\wedge \text{nodes}' = \text{Settle}([\text{nodes} \text{ EXCEPT }$
 $\quad \quad \quad ![n].\text{streams}[m][sid] = \text{ClosedStream},$

$![m].streams[n][sid] = PutReset(@, s.stop)],$
 $n, m, sid)$

— Chain growth —

n finds a new block, giving it something to announce.

$MineBlock \triangleq$
 $\exists n \in InitialPeers :$
 $\wedge Height(n) < MaxBlock$
 $\wedge nodes' = [nodes \text{ EXCEPT } ![n].blocks = @ \cup \{Height(n) + 1\}]$

$Next \triangleq$
 $\vee Connect$
 $\vee OpenHandshakeStream$
 $\vee SendInitResponder$
 $\vee RecvTypeByte$
 $\vee RecvInit$
 $\vee OpenAnnouncementStream$
 $\vee SendAnnouncement$
 $\vee RecvAnnouncement$
 $\vee FinishAnnouncementStream$
 $\vee ResetAnnouncementStream$
 $\vee RecvFin$
 $\vee RecvReset$
 $\vee RecvStop$
 $\vee MineBlock$

Weak fairness on *Next*: as long as any action is enabled, one is taken.
Mining and restarts are bounded, so every behaviour quiesces and every
enabled receive eventually happens. The one loop that is not bounded is a
responder refusing pre-handshake announcement streams while the sender
keeps reopening them; fairness on *RecvInit* guarantees the responder
eventually processes the *init* record that ends that loop.

$Spec \triangleq$
 $Init$
 $\wedge \Box[Next]_{vars}$
 $\wedge WF_{vars}(Next)$
 $\wedge WF_{vars}(RecvInit)$

— Liveness —

Every connection eventually completes its handshake (or has been closed).

HandshakeCompletes $\triangleq$
 $\Diamond \Box \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] :$
 $\text{nodes}[n].\text{conn}[m] = \text{"closed"} \vee \text{HandshakeComplete}(n, m)$

Every peer eventually learns every connected peer's final tip.

AnnouncementsFlow $\triangleq$
 $\Diamond \Box \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] :$
 $\text{nodes}[n].\text{conn}[m] = \text{"closed"} \vee \text{nodes}[n].\text{peer_tip}[m] = \text{Height}(m)$

— Safety invariants —

TypeOK $\triangleq$
 $\forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] :$
 $\wedge \text{nodes}[n].\text{conn}[m] \in \{\text{"none"}, \text{"initiator"}, \text{"responder"}, \text{"closed"}\}$
 $\wedge \text{nodes}[n].\text{close}[m] \in \text{ErrorCodes} \cup \{\text{NoCode}\}$
 $\wedge \text{nodes}[n].\text{version}[m] \in \text{Versions} \cup \{0\}$
 $\wedge \text{nodes}[n].\text{restarts}[m] \in 0 \dots \text{MaxRestarts}$
 $\wedge \forall \text{sid} \in \text{StreamIds}(n, m) :$
 $\wedge S(n, m, \text{sid}).\text{status} \in \{\text{"none"}, \text{"open"}, \text{"closed"}\}$
 $\wedge S(n, m, \text{sid}).\text{rtype} \in \text{StreamTypes} \cup \{\text{"unknown"}\}$
 $\wedge S(n, m, \text{sid}).\text{out} \in \{\text{"open"}, \text{"finished"}, \text{"reset"}, \text{"na"}\}$

A stream slot is free on one side exactly when it is free on the other.

SlotsConsistent $\triangleq$
 $\forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \forall \text{sid} \in \text{StreamIds}(n, m) :$
 $(S(n, m, \text{sid}).\text{status} = \text{"none"}) = (S(m, n, \text{sid}).\text{status} = \text{"none"})$

At most one handshake stream per connection, opened by the initiator

(draft: "Connection Handshake").

OneHandshakeStream $\triangleq$
 $\forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] :$
 $\wedge \text{Cardinality}(\text{HandshakeStreams}(n, m)) \leq 1$
 $\wedge \forall \text{sid} \in \text{HandshakeStreams}(n, m) :$
 $\text{IF } \text{sid}[1] = n \text{ THEN } \text{nodes}[n].\text{conn}[m] = \text{"initiator"}$
 $\text{ELSE } \text{nodes}[m].\text{conn}[n] = \text{"initiator"}$

A peer opens no stream before sending its *init* record, and no announcement stream before its handshake completes (draft: "Connection Handshake").

HandshakeBeforeStreams $\triangleq$
 $\forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] :$
 $\wedge \neg \text{nodes}[n].\text{init_sent}[m] \Rightarrow$
 $\quad \forall k \in 1 \dots \text{MaxStreams} : S(n, m, \langle n, k \rangle).\text{status} = \text{"none"}$
 $\wedge \neg \text{HandshakeComplete}(n, m) \Rightarrow$
 $\quad \forall k \in 1 \dots \text{MaxStreams} : S(n, m, \langle n, k \rangle).\text{rtype} \notin \text{AnnTypes}$

Once n has received m 's *init*, the negotiated version is the minimum of the two advertised versions (draft: "Protocol Versioning").

$$\begin{aligned} \text{NegotiatedVersionIsMin} &\triangleq \\ \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \\ &\quad \text{HandshakeComplete}(n, m) \Rightarrow \\ &\quad \text{nodes}[n].\text{version}[m] = \text{Min}(\text{nodes}[n].\text{my_version}, \text{nodes}[m].\text{my_version}) \end{aligned}$$

A sender never has two announcement streams of one type open toward a peer (draft: "Announcement Streams").

$$\begin{aligned} \text{SenderSingleton} &\triangleq \\ \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \forall t \in \text{AnnTypes} : \\ &\quad \text{Cardinality}(\text{LiveAnn}(n, m, t)) \leq 1 \end{aligned}$$

OBSOLETE is only ever used against a peer that advertised an old version.

$$\begin{aligned} \text{CloseJustified} &\triangleq \\ \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \\ &\quad \text{nodes}[n].\text{close}[m] = \text{"OBSOLETE"} \Rightarrow \\ &\quad \vee \text{nodes}[n].\text{my_version} < \text{MinVersion} \\ &\quad \vee \text{nodes}[m].\text{my_version} < \text{MinVersion} \end{aligned}$$

No interleaving of conformant behaviour ends in a protocol error. There are no adversarial actions in this phase, so any close other than *OBSOLETE* is a disagreement between honest peers about the draft's rules.

$$\begin{aligned} \text{NoHonestProtocolError} &\triangleq \\ \forall n \in \text{InitialPeers} : \forall m \in \text{OtherPeers}[n] : \\ &\quad \text{nodes}[n].\text{close}[m] \in \{\text{NoCode}, \text{"OBSOLETE"}\} \end{aligned}$$